

למה חיפוש "774" לא מחזיר קודם את 774

היום מנוע החיפוש מחזיר את כל מי שהמספר 774 מופיע אצלו — אבל בסדר שרירותי, כך שהרשומה שמספרה בדיוק 774 עלולה להיות בעמוד השלישי. המסמך מסביר למה זה קורה, אילו דרכים קיימות לתקן, ומה כל אחת עולה ביחס למצב היום.

מסמך החלטה קהל: פיתוח + BI נדרשת הברעה: בן

1 · הבעיה, בדוגמה אחת

משתמשת מחפשת אסיר לפי מספר. היא מקלידה 774. באינדקס יש חמש רשומות שהמספר 774 מופיע בהן בצורה כלשהי:

חיפוש 774	
מה מוחזר היום	מה אמור להיות מוחזר
1אבי מזרחי19774מביל	1יוסף לוי774זהה
2שרה בן דוד37742מביל	2דוד כהן7741מתחיל ב־
3דוד כהן7741מתחיל ב־	3משה אברהם77412מתחיל ב־
4יוסף לוי774זהה	4אבי מזרחי19774מביל
5משה אברהם77412מתחיל ב־	5שרה בן דוד37742מביל

אותן חמש תוצאות בדיוק, בסדר אחר. אף אחת לא נעלמת ואף אחת לא נוספת — השאלה היחידה היא מי ראשון. עם דפדוף של חמש תוצאות בעמוד, ההבדל הזה קובע אם המשתמשת מוצאת את הרשומה מיד או מוותרת.

2 · ארבע רמות של "התאמה"

כדי לדבר על העדפה צריך להסכים מה עדיף על מה. בחיפוש הערך 774, רשומה יכולה להתאים באחת מארבע דרכים — מהחזקה לחלשה:

זהה

הערך בשדה הוא בדיוק מה שהוקלד, לא יותר ולא פחות. זו כמעט תמיד הרשומה "774"

מתחיל ב־

הערך פותח במה שהוקלד וממשיך. סביר שהמשתמשת פשוט לא סיימה "7741", "77412" להקליד.

מילה שלמה

מה שהוקלד מופיע כמילה שלמה בתוך הערך. רלוונטי בעיקר לשמות ולשדות "ידי 774 בהן" טקסט חופשי.

מכיל

הרצף מופיע איפשהו בתוך הערך, גם באמצע מספר. הכי רחב, והכי סביר שאינו "37742", "19774" מה שחיפשו.

החלוקה הזו היא כל המסמך. **כל הפתרונות שיוצגו עושים את אותו דבר** — מסמנים לאלסטיק שהרמות האלה אינן שוות ערך. ההבדל ביניהם הוא באמינות הסימון ובמחיר שלו.

3 · למה זה לא קורה מעצמו

התשובה מפתיעה: **אלסטיק לא מדרג את התוצאות האלה בכלל.**

החיפוש היום נשלח בצורה של "מצא כל ערך שמתחיל ב־774" או "שמכיל 774". כששאלתה מנוסחת עם תו כללי כזה, אלסטיק לא טורח לחשב ציון רלוונטיות לכל תוצאה — הוא נותן **לכולן ציון זהה בדיוק**. הסדר שמתקבל בפועל הוא סדר האחסון הפנימי של האינדקס, שאין לו שום קשר לשאלה כמה טוב כל רשומה מתאימה.

זו למעשה בשורה טובה: **הבעיה אינה שאלסטיק מדרג לא נכון, אלא שהוא לא מדרג.** אין כאן אלגוריתם ניקוד להילחם בו — צריך רק להוסיף סימון מפורש של דרגות, וזה בדיוק מה שכל האפשרויות למטה עושות.

נקודה שנייה, פחות אינטואיטיבית: **אלסטיק לא מבחין בין "הערך הוא 774" לבין "המילה 774 מופיעה בערך"**, אלא אם אומרים לו במפורש לבדוק את הערך המלא. לכן "יוסי 774 כהן" עלול להיחשב התאמה מדויקת אם לא מנסחים את השאילתה בזהירות. זה ההבדל בין האפשרויות בסעיף 5, יותר מכל שיקול אחר.

4 · מה יקר ומה זול

אינדקס של אלסטיק בנוי כמו מפתח עניינים בסוף ספר: רשימה ממוינת של כל הערכים, ולצד כל אחד המסמכים שבהם הוא מופיע. מכאן נובע כל ההבדל בעלות:

סוג חיפוש	מה המנדע עושה בפועל	עלות
זהה — 774	קופץ ישירות לערך במפתח. כמו לפתוח מילון בעמוד הנכון.	זניחה
מתחיל ב- — 774*	קופץ ל-774 וקורא את הערכים העוקבים כל עוד הם פותחים באותה קידומת.	נמוכה
מכיל — *774	קורא את כל הערכים במפתח, אחד־אחד, ובודק בכל אחד אם 774 מופיע איפשהו. אי אפשר "לקפוץ" לאמצע ערך.	גבוהה

המסקנה שמשנה את כל הדיון: מה שאנחנו רוצים להוסיף — העדפת התאמה מדויקת — הוא הפעולה הזולה ביותר שקיימת. מה שיקר הוא חיפוש ה"מכיל", **והוא כבר רץ היום**, לפני כל שינוי. סעיף 6 מראה את זה במספרים.

מכפיל אחד שכדאי להכיר: כל אחד מסוגי החיפוש בטבלה מתבצע **בנפרד לכל שדה** שמחפשים בו. חיפוש על 12 שדות עולה פי 12 מחיפוש על שדה אחד — בכל האפשרויות באותה מידה. לכן המכפיל הזה מצטמצם בהשוואה היחסית שבסעיף 6, אבל הוא הנושא של סעיף 7.

חמש דרכים לסמן לאלסטיק שהדרגות אינן שוות. כולן משיגות את התוצאה מסעיף 1; הן נבדלות באמינות, במחיר ובמורכבות. בדוגמאות שלוש שדות חיפוש לשם קריאות — במציאות יש יותר.

שולחים לאלסטיק שאילתה אחת שאומרת "חפש 774, או 774 בהתחלה, או 774 באמצע — ותן למקרה הראשון משקל 1000, לשני 100 ולשלישי 1".

התשאול

```
{
  "query": {
    "query_string": {
      "query": "(774)^1000 OR (774*)^100 OR (*774*)^1",
      "fields": ["fullName", "idNumber", "prisonerNumber"]
    }
  },
  "sort": [ { "_score": "desc" } ],
  "size": 5, "from": 0
}
```

שאילתה אחת, שלוש שכבות בתוך מחרוזת אחת. שום שינוי מבני.

יתרונות

- **שינוי הגדרה בלבד** — שורת טקסט אחת, בלי לגעת במבנה השאילתה.
- אותו חיפוש משמש גם לסינון וגם לדירוג, בלי בדיקה נוספת.
- אפשר לדחוף מהשרת בלי גרסה חדשה של הלקוח.

חסרונות

- **המשקלים לא באמת נשמרים**. למערכת יש כבר משקלים לכל שדה (למשל "שם משפחה חשוב פי 2"), והם מוכפלים במשקלי הדרגות. שדה עם משקל גבוה יכול להקפיץ תוצאת "מתחיל ב-" מעל תוצאה זהה — בדיוק ההיפך מהמבוקש.
- **"זהה" כאן אינו זהה**. גם "יוסי 774 כהן" יקבל את משקל ה-1000, כי אלסטיק בודק מילים ולא את הערך המלא.
- חיפוש של שתי מילים ומעלה מתפרק בצורה שגויה.
- המחרוזת נבנית מטקסט שהמשתמשת הקלידה — גרשיים או סוגריים בשם שוברים את השאילתה.
- **שכבת "מתחיל ב-" כאן אינה חסומה** במספר הערכים שהיא סורקת, ולכן דווקא היא שמייקרת — ראו סעיף 6.

שורה תחתונה הכי מהירה ליישום, אבל לא מבטיחה את התוצאה — ולא בהכרח זולה יותר מ-B. שמישה רק אם מוותרים על המשקלים לכל שדה.

בעברית פשוטה

משאירים את החיפוש הקיים בדיוק כפי שהוא — הוא ממשיך לקבוע מי נכנס לתוצאות — ומוסיפים לצידו שלוש בדיקות שתפקידן היחיד הוא לחלק נקודות: 1000 לזהה, 100 למילה שלמה, 10 למתחיל ב-. הנקודות קבועות ומנותקות לגמרי מחישובי הרלוונטיות של אלסטיק.

התשאול

```
{
  "query": {
    "bool": {
      "must": [
        // קובע מי נכנס
        { "query_string": { "query": "774*",
                          "fields": ["fullName", "idNumber", "prisonerNumber"] } }
      ],
      "should": [
        // קובע רק את הסדר
        { "constant_score": { "boost": 1000, "filter": { "bool": {
          "should": [
            { "term": { "fullName.keyword": "774" } },
            { "term": { "idNumber.keyword": "774" } },
            { "term": { "prisonerNumber.keyword": "774" } }
          ],
          "minimum_should_match": 1 } } } },
        { "constant_score": { "boost": 100, "filter": {
          "multi_match": { "query": "774", "type": "phrase", "lenient": true,
                          "fields": ["fullName", "idNumber", "prisonerNumber"] } } } },
        { "constant_score": { "boost": 10, "filter": {
          "multi_match": { "query": "774", "type": "phrase_prefix", "lenient": true,
                          "max_expansions": 20,
                          "fields": ["fullName", "idNumber", "prisonerNumber"] } } } }
      ]
    }
  },
  "sort": [ { "_score": "desc" },
            { "idNumber.keyword": "asc" } ], // Q4 שובר שוויון, ראו ←
  "size": 5, "from": 0
}
```

הנקודות מצטברות: תוצאה זהה מקבלת 1110, מילה שלמה 110, מתחיל ב- 10, ומכיל 0. הפער קבוע ולכן הסדר לא יכול להתהפך.

יתרונות

חסרונות

- **הסדר מובטח.** הנקודות קבועות, ולכן שום שיקול אחר — משקלי שדות, נדירות מילים — לא יכול להפוך את הסדר.
- **בדיקה אחת יותר מאפשרות A** — אם כי הזולה מכולן.
- דורש שהאינדקס שומר גרסה "לא מפורקת" של כל שדה (keyword). אם היא לא קיימת, שכבת ה"זהה"

- **אף תוצאה לא נעלמת.** הבדיקות רק מוסיפות נקודות, לא מסננות; מספר התוצאות הכולל נשאר זהה.
- "זהה" הוא באמת השוואה לערך המלא, לא למילה בתוכו.
- שכבת "מתחיל ב-" חסומה ל-20 ערכים, ולכן העלות שלה תחומה מלמעלה.
- מוגדר בקוד עם בדיקת טיפוסים, וניתן לכיבוי בנפרד לכל סוג אדם.

פשוט לא עובדת — בשקט, בלי שגיאה. **זה הדבר הראשון שצריך לוודא.**

- **הרבה תוצאות מקבלות ניקוד זהה** (כל ה"מכיל" מקבלות 0). בגלילה אינסופית זה עלול לגרום לרשומה להופיע פעמיים או להיעלם בין עמודים, אם לא מוסיפים שובר שוויון קבוע.
- ההשוואה רגישה לאותיות גדולות/קטנות ולרווחים כפולים, אלא אם מנרמלים באינדקס.

שורה תחתונה הדרך האמינה ביותר, בתוספת עלות שולית ותחומה. זה מה שמיושם היום בקוד.

בעברית פשוטה

החיפוש הרגיל רץ על הכל, ואז בדיקות הדירוג מופעלות רק על 200 התוצאות הראשונות ומסדרות אותן מחדש. במקום לדרג מאה אלף רשומות, מדרגים מאתיים.

התשאול

```
{
  "query": {
    // החיפוש הקיים, ללא שינוי ←
    "query_string": { "query": "*774*",
                      "fields": ["fullName", "idNumber", "prisonerNumber"] }
  },
  "rescore": {
    "window_size": 200, // רק 200 הראשונות מדרגות ←
    "query": {
      "rescore_query": { "bool": { "should": [ ... ] } },
      "query_weight": 1,
      "rescore_query_weight": 1
    }
  },
  "size": 5, "from": 0
}
```

שימו לב: אין `sort` — `rescore` עובד רק כשמיינים לפי ניקוד.

יתרונות

חסרונות

- העלות מנותקת מגודל התוצאה. חיפוש שמחזיר 100,000 רשומות עולה כמו חיפוש שמחזיר 200.
- לא מסתדר עם גלילה אינסופית. מעבר לתוצאה ה-200 הסדר חוזר להיות שרירותי, והמשתמשת רואה "קפיצה" בדיוק בגבול הזה.
- שאילתת החיפוש עצמה לא משתנה בכלל.
- אם כל מה שחשוב הוא שהמדיקות יהיו בעמוד הראשון — זו התשובה המדויקת לצורך.
- מבטל כל מיון מותאם אחר — כולל המיון לפי סקריפט שכבר מוגדר בחלק מסוגי האדם.
- צריך לבחור גודל חלון: גודל מדי מחזיר את העלות, קטן מדי מחמיץ תוצאות.

שורה תחתונה מצוינת אם מסכימים שדירוג מדויק נדרש רק בעמודים הראשונים. פחות מתאימה לרשימה שגוללים עד סופה.

בעברית פשוטה

שולחים שתי שאילתות במקביל בבקשה אחת: אחת זעירה ומיידית ששואלת רק "יש רשומה שהמספר שלה הוא בדיוק 774?", והשנייה היא החיפוש הקיים ללא שינוי כלל. התוצאה המדויקת מוצגת מוצמדת בראש המסך, מעל הרשימה הרגילה.

התשאול

```
POST /asirs/_msearch

{}
{ "query": { "bool": { "filter": { "bool": { // ← filter = בלי חישוב ניקוד כלל
    "should": [
      { "term": { "idNumber.keyword": "774" } },
      { "term": { "prisonerNumber.keyword": "774" } }
    ],
    "minimum_should_match": 1 } } } },
  "size": 3 }

{}
{ ...החיפוש הקיים, מילה במילה, ללא שינוי... }
```

שתי שאילתות בבקשת רשת אחת, מתבצעות במקביל בשרת.

יתרונות

חסרונות

- העומס הנמוך ביותר על השרת מכל האפשרויות — שאילתה שמחזירה תוצאה בודדת בלי חישוב ניקוד.
- דורש שינוי בממשק — אזור "תוצאה מדויקת" נפרד מעל הרשימה.
- החיפוש הקיים לא נוגעים בו בכלל, אז אין סיכון לרגרסיה בביצועים או בתוצאות.
- צריך למנוע הצגה כפולה של אותה רשומה בשני המקומות.
- אפשר להציג את ההתאמה המדויקת מיד — המשתמשת מרגישה מערכת מהירה יותר.
- שני מסלולי קוד לתחזוקה, כולל התנהגות במצב לא־מקוון.
- הדפדוף נשאר פשוט, כי התוצאה המדויקת מחוץ לרשימה הנגללת.
- לא פותר את הסדר בתוך הרשימה עצמה — רק מוציא את המדויקת החוצה.

שורה תחתונה הזולה ביותר והחווייה הברורה ביותר — בתנאי שהעיצוב מאפשר להצמיד תוצאה בראש המסך.

בעברית פשוטה

כותבים קוד קטן שרץ בתוך אלסטיק, מחשב "ציון דרגה" לכל רשומה, וממין לפיו. שליטה מלאה ומפורשת בסדר.

התשאול

```
{
  "query": { "match_phrase": { "idNumber.keyword": "774" } },
  "sort": [
    { "_script": {
      "type": "number", "order": "desc",
      "script": {
        "lang": "painless",
        "source": "def f = doc['idNumber.keyword'];
                    if (f.size() == 0) return 0;
                    def v = f.value; def q = params.q;
                    if (v == q) return 3;
                    if (v.startsWith(q)) return 2;
                    if (v.contains(q)) return 1;
                    return 0; ",
        "params": { "q": "774" }
      }
    } },
    { "_score": "desc" }
  ],
  "size": 5, "from": 0
}
```

הסקריפט רץ מחדש עבור כל רשומה תואמת, לא רק עבור אלה שיוצגו.

יתרונות

חסרונות

- סדר מפורש לחלוטין, בלי להסתמך על מנגנוני ניקוד.
- הכי יקרה בהפרש ניכר — רצה על כל רשומה תואמת, בלי שום קיצור דרך של האינדקס.
- העלות גדלה עם מספר התוצאות, כלומר גרועה במיוחד בדיוק בחיפושים הרחבים שבהם הדירוג הכי נחוץ.
- מיון כזה כבר מוגדר בחלק מסוגי האדם, ובמקומות האלה הוא מבטל כל שכבת דירוג אחרת — נקודה שצריך לבדוק בנפרד.
- אפשר לשלב באותו חישוב שיקולים שאינם טקסטואליים, כמו סטטוס או תאריך.

שורה תחתונה לא מומלץ לצורך הזה. שמור לשיקולי מיון שאי אפשר לבטא כחיפוש רגיל.

6 · כמה כל אפשרות מייקרת את החיפוש

כל המספרים כאן מנורמלים כך שהחיפוש הקיים היום = 100. כלומר "120" פירושו חיפוש שעולה 20% יותר מהיום. מכיוון שכל בדיקה מתבצעת בנפרד לכל שדה, מספר השדות מצטמצם ביחס ואינו משפיע על ההשוואה הזו.

עלות יחסית של כל אפשרות

החלק המלא הוא ההערכה הנמוכה, החלק הבהיר הוא הטווח עד ההערכה הגבוהה. הקו המקווקו הוא המצב היום.

D · חיפוש נפרד 100-101

החיפוש הקיים לא משתנה כלל; השאילתה הנוספת היא חיפוש ערך מדויק בלי ניקוד.

C · דירוג ראש הרשימה 101-108

הכנת שכבות הדירוג משולמת פעם אחת, אבל הן מופעלות רק על 200 רשומות.

B · שכבות ניקוד 106-120

שכבת "מתחיל ב-" חסומה ל-20 ערכים, ולכן הקצה העליון של הטווח תחום.

A · משקלים במחרוזת 106-145

אותה שכבת "מתחיל ב-" כאן אינה חסומה — על שדה עשיר בערכים היא סורקת הרבה יותר.

E · סקריפט 200-400

רץ פר רשומה תואמת — ככל שהחיפוש רחב יותר, כך היחס גרוע יותר.

ויתור על "מכיל" היפדט 15-35

לא אפשרות דירוג אלא נקודת ייחוס מהצורה *10074 ל- *774, בלי שינוי אינדקס.

מה שהגרף מראה: כל אפשרויות הדירוג הסבירות נופלות בטווח 100–145 — כלומר ההבדל ביניהן הוא עשרות אחוזים לכל היותר, והבחירה ביניהן צריכה להיקבע לפי אמינות וחויית משתמש, לא לפי ביצועים.

מה שהגרף מראה חזק יותר: השורה האחרונה. ויתור על חיפוש ה"מכיל" מוריד את העלות לכ־20 — **פי חמישה זול יותר מהמצב היום**, לפני שדיברנו בכלל על דירוג. אם המשתמשות לא באמת מחפשות לפי אמצע מספר, זו ההחלטה בעלת ההשפעה הגדולה ביותר במסמך, והיא לא עולה כלום.

מה בדיוק נוסף בכל אפשרות, ומה מזיז את המספר בתוך הטווח:

אפשרות	מה נוסף מעל החיפוש הקיים	עלות	מה מזיז את המספר
D • חיפוש נפרד	שאלת ערך מדויק, בלי ניקוד, שמחזירה עד 3 תוצאות.	100–101	כמעט כלום. השאלתה הנוספת זולה מכדי להשפיע.
C • דירוג ראש הרשימה	שלוש שכבות, אך מופעלות רק על חלון של 200 רשומות לכל shard.	101–108	גודל החלון. 200 זול, 2000 כבר מתקרב ל-B.
B • שכבות ניקוד	בדיקת ערך מדויק (זניחה) + מילה שלמה (נמוכה) + מתחיל ב־ חסום ל־20.	106–120	שכבת "מתחיל ב־" בלבד. הורדת שכבת "מילה שלמה" חוסכת כ־3.
A • משקלים במחרוזות	שכבת ערך מדויק (מנוקדת ולכן יקרה מעט יותר) + מתחיל ב־ לא חסום .	106–145	כמה ערכים שונים מתחילים ב־774. בשדה מזהים עשיר זה יכול להיות אלפים.
E • סקריפט	הרצת קוד לכל רשומה תואמת, ללא קיצורי דרך של האינדקס.	200–400	מספר התוצאות התואמות. חיפוש רחב = יחס גרוע יותר.
ויתור על "מכיל"	לא מוסיף — מוריד . הפעולה הכבדה ביותר יורדת לגמרי.	15–35	אורך הקידומת. חיפוש בן תו אחד עדיין רחב יחסית.

איך לקרוא את המספרים האלה. הם מודל תיאורטי המבוסס על אופן העבודה של מנוע החיפוש, לא מדידה, ולכן הם מוצגים כטווחים ולא כערך יחיד. הם אמינים לצורך **סדר יחסי** בין האפשרויות — הטענה E יקרה בהרבה מכולן ו־D כמעט חינם" יציבות. הם אינם אמינים כתחזית זמן תגובה. את הערך המדויק אפשר לקבל בהרצה אחת עם דגל הפרופילינג של אלסטיק על האינדקס האמיתי, וזה שווה את הזמן לפני החלטה שדורשת בניית אינדקס מחדש.

7 • שינויים במבנה האינדקס

כל האפשרויות למעלה הן שינויי קוד, וכולן משחקות בטווח שסביב 100. השינויים כאן נוגעים למבנה האינדקס עצמו, דורשים בנייה מחדש שלו — **ולכן הם בתחום ההחלטה של מפתחת ה-BI**. הם גם היחידים שיכולים לשפר ביצועים בסדר גודל, ולא באחוזים.

שינוי	מה זה נותן	המחיר	השפעה
שדה חיפוש מאוחד	בזמן טעינת הנתונים, כל שדות החיפוש מועתקים גם לשדה אחד גדול. החיפוש רץ עליו לבדו במקום על כל שדה בנפרד.	אינדקס גדול יותר; אי אפשר לתת משקל שונה לשדות שונים, ולא לדעת איזה שדה התאים.	הגדולה ביותר
אינדוקס מראש של רצפים	שומרים מראש את כל תתי-המחרוזות, כך שחיפוש "מכיל" הופך לחיפוש רגיל ומיידי במקום לסריקה מלאה.	ניפוח משמעותי של נפח האינדקס וזמן הטעינה.	גדולה מאוד
אינדוקס מראש של קידומות	מייעל חיפושי "מתחיל ב-" בלבד. זול בהרבה מהשורה שמעליו.	תוספת נפח מתונה. לא עוזר ל"מכיל".	בינונית
גרסה "לא מפורקת" לכל שדה	תנאי סף לאפשרויות B, C ו-D. בלי גרסה של הערך המלא, אי אפשר לבדוק שוויון מדויק כלל.	זניח.	נכונות
גרסה "לא מפורקת" לכל שדה	גורם לכך ש"COHEN" ו-"Cohen" ורווח כפול ייחשבו זהים בהשוואה המדויקת.	זניח.	נכונות
מזהים כשדה מזהה, לא טקסט	מספר אסיר, ת"ז וטלפון אינם טקסט חופשי ואין סיבה לפרק אותם למילים. מייתר את כל הבלבול בין "ערך" ל"מילה".	אין; לרוב אף מקטין את האינדקס.	נכונות

אם בוחרים דבר אחד מהטבלה: שדה החיפוש המאוחד. הוא היחיד שתוקף את המכפיל הגדול ביותר במערכת — העובדה שכל חיפוש מתבצע מחדש לכל שדה בנפרד. עם 12 שדות, הוא לבדו שווה יותר מכל ההבדלים בסעיף 6 גם יחד.

8 • מה צריך לברר לפני ההחלטה

שבע שאלות. התשובות לרובן משנות את ההמלצה, לא רק את הניסוח שלה.

Q1

האם לכל שדה חיפוש קיימת באינדקס גרסה של הערך המלא, ולא רק מפורק למילים?

בלעדיה שכבת ההתאמה המדויקת לא עובדת — ובלי להשמיע שגיאה. זהו תנאי סף לאפשרויות B, C ו-D, ולכן השאלה הראשונה.

Q2

האם חיפוש "מכיל" באמת נדרש, או ש"מתחיל ב-" מספיק?

השאלה היקרה ביותר במסמך — לפי סעיף 6 היא שווה פי חמישה מכל בחירה אחרת כאן, ואינה עולה דבר. שווה לבדוק מול המשתמשות אם הן בכלל מחפשות לפי אמצע מספר.

Q3

בכמה שדות מחפשים, וכמה רשומות יש באינדקס?

כל העלויות מוכפלות במספר השדות. בעשרות אלפי רשומות כל האפשרויות מהירות והדין תיאורטי; במיליונים, שדה החיפוש המאוחד הופך לקריטי.

Q4

איזה שדה יכול לשמש "שובר שוויון" יציב במיון?

כל שיטת דירוג בדרגות יוצרת קבוצות גדולות של תוצאות עם ניקוד זהה. בלי כלל הכרעה קבוע, אותה רשומה עלולה להופיע פעמיים או להיעלם בין עמודים בגלילה.

Q5

עד כמה עמוק המשתמשות באמת גוללות?

מכריע בין אפשרות C לאפשרות B. אם כמעט אף אחת לא עוברת את מאתיים התוצאות הראשונות, C זולה יותר ומספקת. בנוסף, דפדוף עמוק מתייקר עם כל עמוד וקיימת תקרה טכנית שכדאי לוודא שלא מגיעים אליה.

Q6

כמה גדולה רשימת ההרשאות (מידור) שנשלחת בכל חיפוש?

אם היא מונה אלפי מזהים, ייתכן שהיא עולה יותר מכל שכבות הדירוג יחד — ואז זו נקודת השיפור האמיתית. יש לכך פתרון מקובל שאינו דורש לשלוח את הרשימה בכל בקשה.

Q7

האם בניית האינדקס מחדש אפשרית, ובאיזה חלון זמן?

מפריד בין מה שאפשר לשלוח מחר לבין מה שנכנס לתכנון. אם בנייה מחדש אינה על השולחן, ההחלטה מצטמצמת ל-A מול B מול D.

9 · המלצה

עבשיר

להישאר ב־אפשרות B, שכבר מיושמת. עלות התוספת היא 6%–20% מעל המצב הקיים, והיא היחידה שמבטיחה את הסדר בלי לשנות את הממשק ובלי לשנות את האינדקס. לצידה להוסיף כלל שובר שוויון קבוע במיון, לפני שהניקוד השוויוני פוגע בגלילה.

לבדוק ראשון

שאלה 1 — האם קיימת באינדקס גרסה של הערך המלא לכל שדה חיפוש. אם לא, שכבת ההתאמה המדויקת אינה עושה דבר כרגע, וזה תיקון מיפוי קטן וזול.

הרווח הגדול

שאלה 2 — לבדוק אם אפשר לוותר על חיפוש ה"מכיל". זה שינוי הגדרה בודד שמוריד את עלות החיפוש לכחמישית, ולפי סעיף 6 הוא שווה יותר מכל שאר ההחלטות במסמך יחד.

אם העיצוב מאפשר

אפשרות D — הצמדת התוצאה המדויקת בראש המסך. העומס הנמוך ביותר על השרת והחווייה הברורה ביותר למשתמשת. שווה לבחון מול מעצבת הממשק.

בטוח ארוך

שדה חיפוש מאוחד באינדקס — הצעד היחיד שמשנה ביצועים בסדר גודל ולא באחוזים, אבל דורש בנייה מחדש ולכן החלטה של מפתחת ה-BI.

לא

אפשרות E (סקריפט) — פי 2 עד פי 4 מהמצב הקיים, ללא יתרון שאין לאחרות. ו**אפשרות C** כל עוד הממשק הוא גלילה אינסופית עד סוף הרשימה.

לצורך דיון מול מפתחת ה-BI, להלן המיפוי בין השמות בעברית לבין המונחים של אלסטיק, ועלות כל אחד. הטבלה היא **לכל שדה ולכל N segment**; מספר השדות, $V =$ מספר הערכים הייחודיים בשדה, $E =$ מספר הערכים שקידומת מרחיבה אליהם, $M =$ מספר המסמכים התואמים.

בעברית	מונח אלסטיק	סיבוכיות	הערה
זהה	<code>term</code> על <code>keyword</code>	$O(\log V)$	שדה שאינו ממופה מחזיר תוצאה ריקה מיידית, ללא שגיאה.
מילה שלמה	<code>match_phrase</code>	$O(\text{postings})$	למונח יחיד מתנוון ל- <code>match</code> רגיל.
מתחיל ב- (חסום)	<code>match_phrase_prefix</code>	$O(E \cdot \text{postings})$	E חסום ב- <code>max_expansions</code> . קטיעה פוגעת בדירוג בלבד, לא ברלוונטיות.
מתחיל ב- (לא חסום)	<code>prefix</code> / <code>x*</code> ב- <code>query_string</code>	$O(E \cdot \text{postings})$	E אינו חסום — זה מקור הקצה העליון בטווח של אפשרות A.
מכיל	<code>wildcard</code> מוביל <code>*x*</code>	$O(V)$	סריקת מילון מונחים מלאה. ניתן להאצה רק במיפוי.
סקריפט	<code>sort _script</code>	$O(M)$	ללא האצת אינדקס וללא <code>early termination</code> .

בסיס הנרמול בסעיף 6: `query_string = 100` עם `*774*` על N שדות, כלומר N פעמים $O(V)$. כל התוספות נמדדו כיחס לאותו בסיס, ולכן N מצטמצם.

שמות האפשרויות במונחי אלסטיק: **A** = בוסטים ב-`query_string`; **B** = `bool.should` עם `constant_score` (`function_score`) עם `score_mode: max` היא וריאציה שקולה בעלות); **C** = `rescore` עם `window_size`; **D** = `_msearch` עם שאילתת `term` ב-`sort _script`; **E** = `filter context`.

שינויי המיפוי: שדה מאוחד = `copy_to`; רצפים = `ngram analyzer` או טיפוס שדה `wildcard`; קידומות = `index_prefixes` או `search_as_you_type`; ערך מלא = subfield מסוג `keyword`; נרמול = `normalizer`. רשימת המידור = `terms` filter, ופתרון הגודל הוא `terms_lookup`. דפדוף עמוק = `search_after` במקום `from`, ותקרת `index.max_result_window` היא 10,000 כברירת מחדל.

הערכות העלות במסמך הן מודל תיאורטי המבוסס על אופן העבודה של מנוע החיפוש, לא מדידה, ומוצגות כטווחים מסיבה זו. הן אמינות לצורך השוואה יחסית בין האפשרויות ולא כתחזית זמן תגובה. לפני החלטה שדורשת בניית אינדקס מחדש, הרצה אחת עם דגל הפרופילינג של אלסטיק תיתן פירוק זמנים אמיתי ותכריע את הדיון במספרים.